

Курсов проект финален изпит

Описание на проекта

Образователна институция иска да разработи система за управление на основните си академични процеси – студенти, курсове, преподаватели, катедри и извънкласни дейности. Системата трябва да предоставя RESTful API за управление и търсене на данни, като спазва добрите практики за слоеста архитектура и работа с база данни.

Проектът представлява Campus Management System, реализиран като Spring Boot приложение.

Бизнес спецификация

Системата трябва да поддържа следните основни функционалности:

- Управление на студенти и техните лични данни;
- Управление на адреси на студенти;
- Управление на курсове и катедри;
- Управление на преподаватели;
- Записване на студенти в курсове;
- Управление на студентски клубове;
- Търсене на студенти по различни критерии, включително такива от свързани ентитита.

Основни ентитита и релации

Ентитита (минимум 7)

- Student – студент;
- Address – адрес на студент;
- Course – курс;
- Department – катедра;
- Professor – преподавател;
- Enrollment – записване на студент в курс;
- Club – студентски клуб.

Релации между ентитетата

- Student ↔ Address – One to One;
- Student ↔ Enrollment – One to Many;
- Enrollment ↔ Course – Many to One;
- Enrollment ↔ Student – Many to One;
- Department ↔ Course – One to Many;
- Department ↔ Professor – One to Many;
- Student ↔ Club – Many to Many.

Всички релации трябва да бъдат коректно описани чрез JPA анотации и ясно документираны в кода.

Търсене (Dynamic Search)

Трябва да бъде реализирано динамично търсене на студенти (или друг основен ентити) чрез Filter DTO.

Пример: `StudentFilterDTO`

Филтърът трябва да съдържа минимум 5 полета, например:

- име на студент;
- град (от Address);
- име на клуб (от Club);
- име на курс (от Enrollment / Course);
- година на записване (от Enrollment).

Технически изисквания за търсенето

- Реализацията да бъде чрез:
 - CriteriaBuilder, или
 - JPA Specification, или
 - Criteria API;
- Минимум едно поле трябва да бъде от асоциирано ентити;
- Да бъде предоставен примерен клас за Filter DTO и примерна реализация на динамичното търсене.

REST API – Основни endpoints (примерни)

Функционалност	HTTP метод	Endpoint	Описание
Създаване на студент	POST	<code>/api/students</code>	Създава нов студент
Редакция на студент	PUT	<code>/api/students/{id}</code>	Редактира данни за студент
Изтриване на студент	DELETE	<code>/api/students/{id}</code>	Изтрива студент
Списък с курсове	GET	<code>/api/courses</code>	Връща всички курсове
Записване в курс	POST	<code>/api/enrollments</code>	Записва студент в курс
Списък със студентски клубове	GET	<code>/api/clubs</code>	Връща всички клубове
Търсене на студенти	POST	<code>/api/students/search</code>	Динамично търсене

Реалният набор от endpoints е по избор на студента, но трябва да покрива цялата бизнес логика.

Архитектура и слоеве

Проектът трябва да следва стриктно слоева архитектура:

- Controller layer – обработка на HTTP заявки и валидации;
- Service layer – бизнес логика;
- Repository layer – достъп до база данни чрез Spring Data JPA.

Изисквания към проекта

- Проектът трябва да бъде разработен със Spring Boot;
- Задължително използване на Spring Data JPA;
- Използване на релационна база данни – H2, PostgreSQL или MySQL;
- Създайте отделни класове за Controller, Service и Repository за ентититата
- Спазване на Java конвенциите за именуване на кода;
- Капсулация и ясно разделение на отговорностите;
- Отделни класове за Controller, Service и Repository за всяко основно ентити.
- Валидации на входните данни в Controller слоя;
- Коректна обработка на грешки и подходящи HTTP статус кодове;
- Всички релации и ключови методи трябва да бъдат документиран;
- Кодът трябва да бъде структуриран и четим;
- Проектът трябва да може да бъде стартиран локално без допълнителна конфигурация;
- Кодът трябва да бъде предаден като линк към GitLab - а на факултета, в съответното задание в Google Classroom

Формиране на крайната оценка

По време на изпита студентите трябва да демонстрират:

- работещо REST API;
- коректна работа с база данни и релации;
- динамично търсене с Criteria / Specification;
- спазване на архитектурните принципи;
- чист и добре структуриран код.